Arhitecturi:

- SISD (Von Neumann)
- SIMD
- MISD
- MIMD (procesoare de azi)

Shared memory:

- EREW – Exclusive Read Exclusive Write
- CREW – Concurrent Read Exclusive Write - cel mai des întâlnit
- ERCW
- CRCW

Mutex: Dekker, Peterson(Busy wait), Dijsktra

Complexitati

S = G / T

C = T * P

E = G / C = G / TP = S / P

Amdahl: T = fG + (1 - f)G/P, f = procentul de operatii neparalelizabile din algoritm

Bubble Sort <-> OETS (Odd-Even Transposition Sort)

In paralel, se compara pozitiile: (1, 2), (3, 4), ...,apoi (2, 3), (4, 5),.....

Complexitate initiala: O(n^2)

Complexitate paralela: O(n^2 / p), pentru n = p este O(n)

Speedup = (de fapt) (P logN) / N

Shear Sort

La fiecare pas, se sorteaza coloanele impare cresc, iar cele pare descresc., apoi sesorteaza crescator coloanele.

Complexitate: G = O (N * log2N)

T = N/P logN * log2 sqrt(N)

S = P / log2 (sqrt N)

Rank Sort

Pentru fiecare element se pastreaza cate elemente sunt mai mari decat el, apoi e pun in altvector elementele, in ordine, dupa acest criteriu.

Complexitate: G = O(n^2)

T = O(n^2) / p

S = log/n * p

Eficienta = logn/n

Multiplicare matrice

S = T/G = N^3 / (N^3 / P) = P

Producatori – consumatori

M producatori si N consumatori

Producatorii: 1 mutex si 1 semafor

- produc
- daca au loc, pun produsul
- semnalizeaza ca au ocupat spatiul

Consumatorii: 1 mutex si un semafor

- intra in locul ocupat de produs
- iau produsul
- semnalizeaza ca spatiul e gol

Problema filozofilor

Toti filozofii, in afara de ultimul:

- ocupa batul din stanga si cel din dreapta
- mananca
- elibereaza betisoarele

Ultimul filozof:

- ocupa batul din dreapta, apoi cel din stanga
- mananca
- elibereaza betisoarele

Cititori – scriitori

2 solutii: excludere mutuala -> prioritate pentru cititori

Sincronizare conditionata -> prioritate pentru scriitori

1) excludere mutuala

Cititori:

- daca nu mai citeste alt cititor, blocheaza zona comuna
- citeste
- daca e singurul cititor, deblocheaza zona comuna

Scriitori:

- blocheaza zona comuna
- scrie
- deblocheaza zona comuna

- scriitori pot astepta f mult -> writer's starvation
- 2 sau mai multi scriitori nu pot scrie in acelasi timp

2) Sincronizare conditionata

Cititori:

- Asteapta pana nr_writers = 0, apoi creste nr_readeri
- Citeste din resursa comuna
- Scade nr de readeri

Scriitori:

- Asteapta pana nr_writers = 0 si nr_readeri = 0 si creste nr_writeri
- Scrie in resursa comuna
- Scade nr de writeri

Problema barbierului

Nr_scaune = n;

Semafoare pentru: scaune, clienti si pentru verificare daca barbierul este gata de tuns

Barbier:

- Blocheaza zona de clienti
- Blocheaza zona de scaune ---------- |
- Incrementeaza nr de scaune libere |
- Deblocheaza Barbier_gata |
- Elibereaza zona de scaune----------|
- tunde

Clienti:

- Blocheaza zona de scaune ----------|
- Daca e loc, decrementeaza nr de scaune |
  - Deblocheaza zona de clienti |
  - Blocheaza barbier_gata |
  - Este tuns |
- Elibereazazona de scaune----------|

Sortare cu pipeline

- Fiecare celula primeste un numar la stanga, si da la dreapta maximul dintre nr primit si cel pe care il tine
- Fiecare celula transmite de la dreapta spre stanga.

N valori:

➔ 2n-1 pasi in prima faza
➔ 4n-3 sau 3n-2 pasi pentru a doua faza

Binary search / Parallel search

Complexitate: G = O(log2 N)

T = O(log p+1 (n + 1))

- Toate elementele trebuie sa fie distincte

Sume prefix

- Complexitatea: O(logN)
- SIMD – fara bariera
- MIMD – 2 bariere

Parallel Scan – O(log(n)) cu arbore

Efficient Value Broadcast

- Liniar: O(n)
- Paralel cu p = n/2: O(log^2 N)

Modelul Foster

- Tcalcul – depinde de dimensiune problema, nr taskuri, proc. Si mem.
- Tcommun – inter-proc si intra-proc
- Tidle – din cauza lipsei calculelor sau datelor
- T = Tcalcul + Tcommun + Tidle

Floyd paralel:

Tf1 = tc * N^3 / P + N*logP*(ts + tw*N), P <=N

Tf2 = tc * N^3 / P + N*logP*(ts + tw*N / sqrt(P)), pana la N^2 procesoare

Model LogP

Latency – intarziere sursa->destinatar

Overhead – durata a procesului de a primi sau trimite un msj

Gap – interval minim intre 2 transmisii sau receptii ale unui modul

Po->p1 – timp_total = overhead_transm. + latenta + overhead rec.

Ceasuri fizice

Clock skew - diferenta de timp intre calculatoare

- Nu poate fi eliminat complet

Flaviu cristian – time server, cu care se sincronizeaza celelalte ceasuri

Berkeley – un server afla timpii clientilor si face media (le pune de acord)

Network time protocol (descentralizat) – straturi:

1. Ceasuri de referinta (atomic clocks, US naval observatory)
2. Routere, servere imp.

- Protocol NTP = f complex
- Sincronizare ceasuri – 1-50 ms

Ceasuri logice – Lamport

- Relatia "->" (petrecut inainte) – foarte logic
- Este importanta ordinea operatiilor, nu timpul exact

Reguli:

- Variabila initiala 'cl = 0 (initial)' pentru fiecare proces, si mesajele au un camp 'tt';
- La transmitere: cl++, tt = cl;
- La primire: cl = max(cl, tt) + 1;

a -> b ==> cl(a) < cl(b)

DAR: cl(a) < cl(b) =\=> a -> b

Ceasuri logice vectoriale

Fiecare Pi are un tablou Vi unde Vi[k] = numarul de evenimente despre care stie ca au avut loc la Pk.

Pi transmite mesaj 'm' ==> adauga Vi la 'm'

Pj primeste mesaj 'm+Vi' si ajusteaza: Vj[k] = max (Vj[k], vtm[k]), pt fiecare k, apoi Vj[i]++

Algoritmi unda

Proprietati: terminare, decizie, dependenta

Categorii: initiatori, neinitiatori(primul eveniment este un receive)

Sensul legaturilor:

- Inel: 2 directii (precedent si urmator)
- Clica: n noduri de n-1, directia de la i la j are sensul (j - i) mod n
- Hipercub: n dimensional, nodurile (b0,b1...bn-1),(c0,c1...cn-1) legate cu eticheta i difera la bitul i

Algoritm inel:

- Initiatorul trimite un token
- Token estepasat pana ajunge inapoi la initiator
- Initiatorul ia decizia
- Mesaje= n, Timp = n

Algoritm arbore:

- Frunze -> radacina
- Fiecare nod cunoaste vecinii, si stie daca a primit msj de la un vecin
- Fiecare proces trimite exact un mesaj
- Daca a primit de la toate canalele mai putin 1, trimite mesaj pe el
- Daca a primit de la toate canalele, atunci decide(inseamna ca el e radacina)
- Mesaje = n, Timp = O(D)

Algoritm ecou:

- Radacina -> frunze -> inapoi radacina
- Topologii arbitrare
- Un singur initiator (e centralizat)
- Inundarea retelei cu mesaje ‘tok’
- Mesaje = 2|E|, Timp = O(N)

Algoritmul fazelor – algoritm unda:

- Descentralizat
- Topologii arbitrare
- Canale unidirectionale ==> avem Vecini-in si vecini-out
- Mesajul i+1 se transmite fiecarui out-vecin dupa ce am primit mesaj i de la fiecare in-vecin
- Mesaje = 2D|E|, Timp = 2D, D= diametru graf
- Pentru clici: Mesaje = N(N - 1), Timp = 2

Algortimul lui Finn – algoritm unda:

- Mesaje <= 2N|E|, Timp = O(D)

Algortim arbore pentru alegerea liderului:

- Mesaje = 4n – 4 = O(N)
- Timp = 3D + 1 pasi = O(D)

Algoritmul LeLann – alegere lider:

- Fiecare proces difuseaza id-ul sau
- Fiecare porces colecteaza numerele celorlalti
- Fiecare proces calc. maximul, procesul cu nr maxim devenind liderul
- Mesaje = O(N^2)
- Timp = O(N)

Algoritmul Lelann-Chang-Robert – alegere lider:

- Fiecare proces trimite la dreapta Id-ul sau
- Fiecare proces primeste mesaj m:
  - M > id: trimite m
  - M < id: sterge m
  - M = id: lider
- Doar procesul cu Id maxim termina; poate trimite mesaj special pt a debloca restul

  Modificat:
  - cel putin un proces initiaza si se marcheaza
  - un proces nemarcat primeste mesaj -> genereaza mesaj cu id-ul lui, se marcheaza si continua cu procesare mesaj m

- Timpul: O(n)
- Mesaje: minim 2n-1, maxim n(n+1)/2

Algoritmul Hirschberg-Sinclair:

- Mesaje: O(N logN)
- Timp: O(n)
- Inel bidirectional
- Instructiuni: sendboth, sendpass, sendecho

- In faza k, un proces initiaza mesaje pe ambele directii pe cai cu lungime 2^k
- Procesele din cale:
  - Decide ca nu poate castiga alegerea -> face pass si nu initiaza mesaje proprii
  - Decide ca sursa nu poate castiga -> transmite ecou negativ
  - Ultimul proces din calea 2^k transmite ecou pozitiv in loc de pass
  - Daca primeste mesaj cu id propriu, a castigat si trimite mesaj sa anunteca a castigat

Difuzari:

- Cu sondaje: N-1 mesaje
- Prin inundare: 2M mesaje, M = nr legaturi

Algoritmul pulsatiilor:

- Fiecare nod trimite vecinilor matricea sa de adiacenta si o primeste pe a vecinilor
- Dupa R runde, stim legaturile posibile alenodurilor la distanta <=R
- Nasol ca nu stim diametrul retelei + trafic inutil

Algoritmi cu mesaje de sondaj si ecou:

- Faza 1 – mesaje de sondaj - radacina --> frunze
- Faza 2 – mesaje de ecou – frunze --> radacina
- Fiecare nod:
    - Primeste mesaj de sondaj
    - Propaga mesajul celorlalti vecini si asteapta ecouri (pe aceleasi canale)
- 2 * nr_legaturi_prima_sonda + 4 * restul_legaturilor + N (transmitere topologie)

Terminare programe distribuite - arbore:

- Cand o frunza termina, anunta parintele
- Cand unpoces oarecare se termina, asteapta notificare de la fii, apoi anunta parintele
- Mesaje: radacina-> frunze, confirmare: frunze -> radacina

Algoritmul Huang:

- Procese: active sau libere
- Doar procesele libere dau mesaje
- Proces liber --> proces activ – la primirea unui mesaj de calcul
- Terminarea: cand sunt doar procese libere

Problema generalilor bizantini:

- Regula 1: toti locotenentii loiali se supun aceluiasi ordin
- Regula2: daca comandantul e loial, fiecare locotenent loial se supune ordinului comandantului
- Evenimente:
    - Comandant trimite ordin -> locotenenti
    - Unlocotenent trimite -> ceilalti locotenenti – mesajul de la comandant
    - Un locotenent decide prin vot majoritar ce decizie ia